GROUP PROJECT

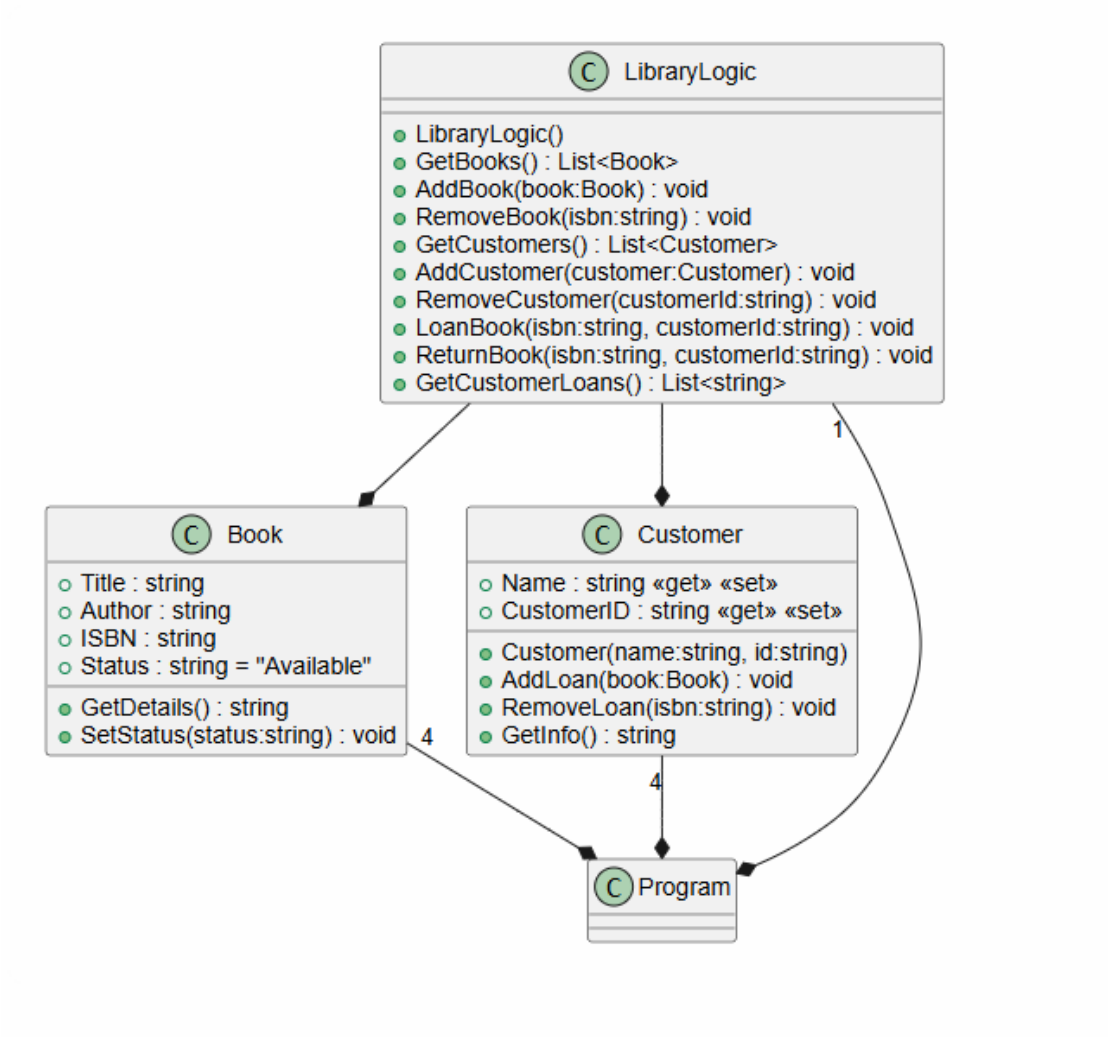
Library Management System in C#

John Andersson, Magnus Bodin, Evgenia Karveli

1. Solution Description

The application works as a library management for librarians. You can add a book, loan a book and check if the book is on loan or available. You can also see customer names and their loans. LibraryLogic class is connected with the Book and Customer class.

2. UML Class Diagram



3. Work Division

Since our first meeting, we decided to divide the Library Management System into distinct modules based on the three required C# classes for a smooth development process. More specifically, John was responsible for the Book Class. His focus was on

the library's inventory, including titles, authors, ISBNs, and availability status. Magnus was responsible for the LibraryLogic Class, which handles the interactions between books and customers. Evgenia was responsible for the Customer Class, including the definitions of customer attributes such as ID and Name, as well as any methods related to customer profile management.

4. Meeting Minutes

In total, we held four Zoom meetings, as detailed below.

- Meeting 1 - Date: Friday, 24/04, Time: 16:00, Duration: 15 min

In this meeting, we basically defined the project scope and we assigned tasks. We wanted to have a shared understanding of the technical requirements of the assignment, and we made the work division as mentioned above. On 30/04, Magnus initialized a public GitHub repository named "LibraryManagementAssignment", which allowed us to begin working in parallel using different branches.

- Meeting 2 - Date: Tuesday, 05/05, Time: 16:00, Duration: 30 min

Each one of us presented our current branch progress, and we collectively addressed logic errors.

- Meeting 3 - Date: Friday, 08/05, Time: 16:00, Duration: 30 min

We used the feedback from the last meeting to make some changes and present them to the group. We discussed what we had still to do.

- Meeting 4 - Date: Tuesday, 12/05, Time: 16:00, Duration: 70 min

During this session, we integrated all components into Program.cs. We had encountered some difficulties in keeping track of all the changes and we had exchanged several messages on Canvas to coordinate and improve the code. During the meeting, we spent most of the time working on the Console Menu and handling some language inconsistencies between classes. We shared our final thoughts to confirm the project met all requirements. Lastly, Evgenia took on the task of creating and uploading to GitHub a Word document for the final report with the details of each of our meetings for sections 3 and 4. John was also assigned to create the UML Class Diagram.

- Meeting 5 – Date: Monday, 18/05, Time: 20:00, Duration: 45 min

This was our last meeting in which we discussed the final details of our report before the submission of our project.

5. Individual Contribution Log

- John Andersson

Weekly Task Log

Week: 18: First, I looked at all the classes and how they would work together in an application. When our group had assigned who would do which class I started creating my class which was the book class.

Week :19: Continued on building the book class which had 4 properties, Title, Author, ISBN and Status. Created two methods that were GetDetails() and SetStatus(). GetDetails() handles the book information from user input and gets added into a list. SetStatus() handles if the book is available or on loan.

Week: 20: Started putting my parts together in program.cs This included to register a book and adding them to a list. Also did the remove a book and fixed some errors regarding input validation so that the application won't crash and would handle user input correctly.

File/Line Attribution

File name: Program.cs - Lines contributed: Line 134-189. Including registering a book with a unique ISBN that's generated randomly with 13 numbers also includes removing a book from the system by using/asking for ISBN. Both handle input validation.

File name: Book.cs - Lines contributed: Line 1-35. This includes the class book. Which handles strings such as title, author, isbn. Two key methods, GetDetails() and SetStatus().

Problem log/solved.

CS1503 argument 1 cannot convert from random to string. Was caused by the random generating of isbn. Fixed it by making isbn an empty string. **string isbn = "";**

CS0103, the name does not exist in current context - Caused by misplaced curly brackets. Fixed the issue by finding it and removing and placing them where they needed to be.

Class interaction with the application. I first made my own list to test, register and remove books. When I added it to the project, it did not work because Library Logic already used a different list. My teammate noticed it, and I fixed it by using the existing list instead.

- Magnus Bodin

Weekly Task Log

Week 18 - This week was all about learning GitHub and trying to set up a shared repository. I created an empty C# console project with the three classes and the Project.cs file and uploaded the empty files to the remote repository for the group to use.

Week 19 - Started coding the LibraryLocic class file and the Main() method and made the first ”real” commit/push to GitHub.

- GetBooks() and GetCustomers() are identical and simple to program since all they do is return a list. In the Main() method I added a menu (options 0-10) and two corresponding methods (1 and 4) that prints the returned lists. Both the menu and the print-methods are reused code from the PlantCare assignment.
- AddBook() and AddCustomer() are identical so the work flow is the same - take an object as an argument - check if it’s already in the list - if it is, print info and return - if not, add to list.
- RemoveBook() and RemoveCustomer() are also identical in that they both take an unique identifier as argument - check if the object is in the list - if no, print info and return - if yes, check if book is ”on loan” or if customer have loans - if yes, print info and return - if no, remove from list.
- LoanBook() takes two identifiers as arguments - assigns an empty book and customer the values of those objects (if they are found in the lists) - handles exceptions if not found in lists or if book is already on loan - if all is ok, calls SetStatus() and AddLoan() from the Book() and Customer() classes. Corresponding method LoanBookToCustomer() (7) was added to the Main method, asking the user for the book and customer id:s.
- ReturnBook() also takes two identifiers as arguments and, after finding the customer - searches through the customers loanedBooks list and, if all is ok - calls SetStatus() and RemoveLoan() from the Book() and Customer() classes. Corresponding method ReturnBook() (8) was added to the Main() method asking the user for the book and customer id:s.

Week 20 - After every member of the group added their code to the repo the work started with trying to get the program to run. A lot of changes were done, mostly to property names and what arguments to send to the methods. A group decision was made on what the method GetCustomerLoan() should do and I wrote code so that the method goes through every customers LoanedBooks lists and adds the info to a new list that is

returned. In the Main()method the corresponding method DisplayBooksOnLoan() (10) was added that prints out the returned list of strings.

Problems, design and methods

In all methods searching through lists I decided to use simple foreach-loops and if-statements instead of using the more advanced linked LINQ methods. Mostly because it's easier to see what they do and fix errors.

Before we all shared our code on GitHub I had AI make temporary Book and Customer classes so that I had something to test my methods with.

- Evgenia Karveli

I was responsible for designing the Customer class in Customer.cs and I also worked on Program.cs, where I added the main menu's customer functionality.

The first week, I focused on teamwork discussing the design of the system. I worked with my colleagues on how the classes would connect to each other and we decided in common our tasks. Then, I started writing the code for the Customer class. I made a list collection to follow the Book objects that each customer checked out, and I used encapsulation to protect the object data using public properties with get/set accessors. I also wrote a special method called GetInfo() to return a formatted string of this data to the console. This method acts as a profile summary as it pulls the customer's details and executes a foreach loop over their active loans to append every borrowed book title into a string. I included conditional logic for empty states, and the system appends a "None" label instead of leaving the output blank or throwing a runtime error, in case that a customer has no active loans. To avoid confusing my teammates' code, I created a separate GitHub branch called final_customer_fix. So, I safely linked my code to theirs and opened a pull request to merge everything without conflicts. Last week, I combined everything in the main menu by enabling the registration and removal options. I wrote first the RegisterCustomer() method for creating which uses string.IsNullOrWhiteSpace to block empty console inputs and guard against bad data entering the system before making a new Customer object to the system. The RemoveCustomer() method checks user's inputs against existing IDs and guides the user with status labels like [SUCCESS], [WARNING], and [ERROR] for success and error states.

One of the practical problems I encountered was when I tried to combine our code and the Customer class couldn't read the Book data because the property names didn't

match and some were in uppercase and some in lowercase. However, I fixed this by updating the code and in cooperation with my group. Specifically, I used the expression b => b.ISBN == isbn and a nullable reference type (Book?) to make the RemoveLoan method to accept a string parameter and locate and isolate the target book. I also noticed a logic error. If we deleted a customer from the system, any books they had borrowed would disappear with them and remain locked forever. For this reason, I added a security check in RemoveCustomer() that evaluates the LoanedBooks.Count and blocks the deletion and the database transaction warning the user if a customer still owed books.

6. Screenshots of testing the System

Screenshot 1. Adding and deleting books from system.

```
// Test 1. Try to remove customer that does not exist
Console.Write("Test 1: " );
library.RemoveCustomer("009");          // Output: customer not found

// Test 2. Try to remove book that is "On Loan"
Console.Write("Test 2: " );
library.RemoveBook("9780241383476");        // Output: book is on loan

// Test 3. Try to remove book with false ISBN
Console.Write("Test 3: " );
library.RemoveBook("0000000000000");        // Output: book not found

// Test 4. Try to add a new book with ISBN that is already in the library
Console.Write("Test 4: " );
var book5 = new Book("Blodapelsin", "Conny McDonald", "9780679641041"); // Same ISBN as book4
library.AddBook(book5);                     // Output: book already exists
```

Screenshot 2. Code for testing.

```
Test 1: Customer not found.
Test 2: Book is on loan.
Test 3: Book not found.
Test 4: Book already exists.
```

Screenshot 3. Error handling.